Name : Nicholas Brandon Chang
NIM : 2702288065
Topic: AOL Data Structure (Documentation)

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

Explanation : All of the library function that I use for this code contains the standard input output library, string.h for string manipulation and stdlib.h for accessing memory allocation.

```
struct trie{
   char character; //Each node stores a character
   int Word; //to mark the end of a word
   char *desc; //a description of the slang word
   struct trie *child[128]; //
};
```

Explanation: This is a structure that represents a node in the Trie algorithm.

```
void insertWord(struct trie **root, char *word, char *desc){
   if(*root == NULL) *root = createNode('*');

   struct trie *curr = *root;
   while(*word) {
      if(curr->child[*word] == NULL){
         curr->child[*word] = createNode(*word);
      }
      curr = curr->child[*word];
      word++;
```

```
    }
    curr->Word = 1;
    curr->desc = malloc(strlen(desc) + 1);
    strcpy(curr->desc, desc);
}
```

Explanation: This function inserts a new word into the Trie. If the Trie is empty, it creates a new root node. It then travels from node to node according to the characters in the word creating new nodes. When it reaches the end of the word, it serts the word and copies the description into the node.

```
void printTrie(struct trie *node, char *buffer, int depth, int *count){
    int i;
    if(node == NULL) return;

    if(node->character != '*'){
        buffer[depth] = node->character;
    }

    if(node->Word){
        buffer[depth + 1] = '\0';
        printf("%d. %s\n",(*count)++, buffer);
    }

    for(i = 0; i < 128; i++){
        if(node->child[i] != NULL){
            if(node->character != '*') depth++;
            printTrie(node->child[i], buffer, depth, count);
        }
    }
}
```

Explanation: This functions prints all the words in the Trie. It uses a DFS or depth first search to travel the Trie and a buffer or u could call it temp to store the current word. When it reached a node that represents the end of a word (EOW) it then prints the word.

```
void searchPrefix(struct trie *root, char *prefix){
   int i;
   char buffer[100];
   int depth = 0;
   int count = 1;
   struct trie *curr = root;

   while(*prefix){
      if(curr == NULL) return;
      curr = curr->child[*prefix];
      buffer[depth++] = *prefix++;
   }

   if(curr == NULL){
            printf("There is no prefix ""%s"" in the dictionary.\n", buffer);
            return;
            }

   if(curr != NULL){
      buffer[depth] = '\0';
      printf("Words starting with ""%s"": \n", buffer);
      if(curr->Word){
         printf("%d. %s\n", count++, buffer);
      }
   }
```

```
    for(i = 0; i < 128; i++){
        if(curr->child[i] != NULL){
            printTrie(curr->child[i], buffer, depth, &count);
        }
    }
}
```

Explanation: This function prints all words in the prefix same as the previous function but what makes it different is that is starts with a given prefix. It travels the Trie nodes to the end of the prefix, the calls printTrie again to print all the words that branch from that prefix node.

```
void exists(struct trie *root, char *word){
            char *saveWord = word;
    struct trie *curr = root;
    while(*word){
        if (curr == NULL || curr->child[*word] == NULL) {
            printf("The word %s is not present in the dictionary.\n", word);
            return;
        }
        curr = curr->child[*word];
        word++;
    }
    if(curr != NULL && curr->Word){
        printf("Slang word: %s\n", saveWord);
        printf("Description: %s\n", curr->desc);
    }
}
```

Explanation: This function I use in the search function but not the searchprefix menu it travels the Tire according to the character instead of the prefix. Then if it reached the EOW or end of word and the

current node is the end of the word, that means the word exists and I print the word and the description too.

```
int haveSpace(char *str){
   while(*str){
      if(*str == ' ') return 1;
      str++;
   }
   return 0;
}
```

Explanation: This function is just too check the spaces in the string because the case has several conditions about the input.

```
int wordCount(char *str){
   int count = 0;
   int inWord = 0;
   while(*str){
      if(*str == ' '){
         inWord = 0;
      }else{
         count += !inWord;
         inWord = 1;
      }
      str++;
   }
   return count;
}
```

Explanation: This functionis for counting the number of words in a string for my description menu because it can only be inputted if it has more than one word.

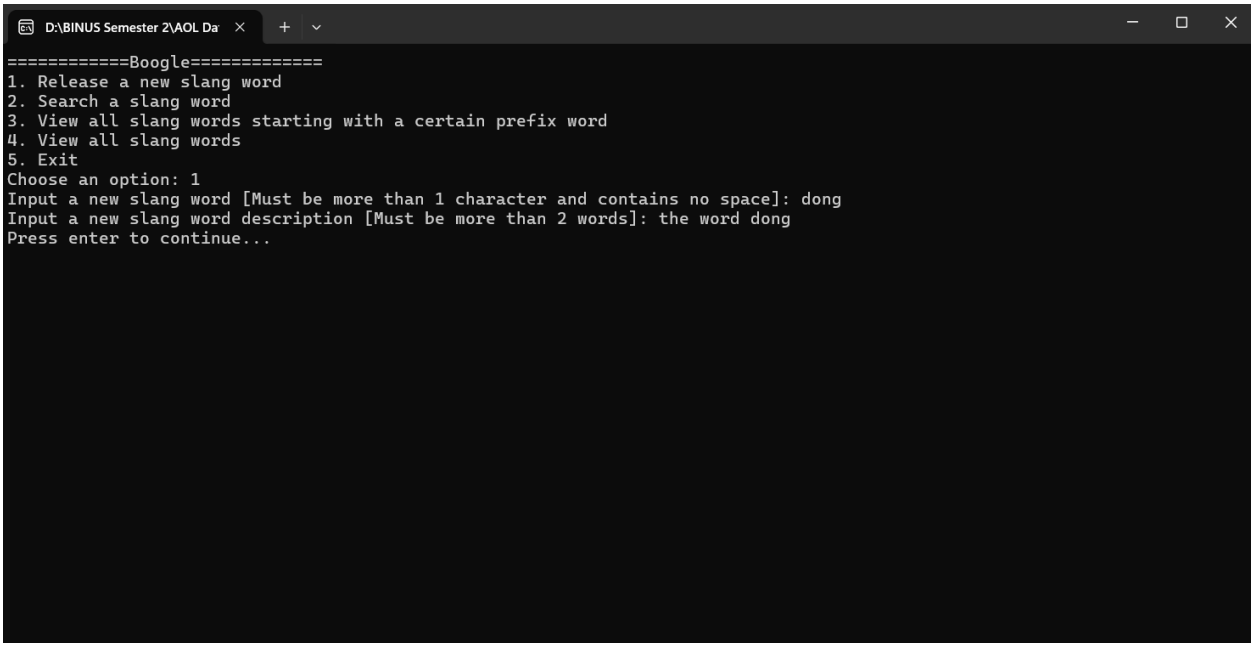
```
int main(){
   struct trie *root = NULL;
   char word[100];
   char description[100];
   int option;
   int count = 1;

   while(1){
         system("cls");
         printf("============Boogle=============\n");
      printf("1. Release a new slang word\n");
      printf("2. Search a slang word\n");
      printf("3. View all slang words starting with a certain prefix word\n");
      printf("4. View all slang words\n");
      printf("5. Exit\n");
      printf("Choose an option: ");
      scanf("%d", &option);getchar();
      switch(option){
         case 1:
            do{
               printf("Input a new slang word [Must be more than 1 character and contains no space]: ");
               scanf("%[^\n]", word);getchar();
            }while(strlen(word) <= 1 || haveSpace(word));

                                    do{
                  printf("Input a new slang word description [Must be more than 2 words]: ");
                  scanf("%[^\n]", description);getchar();
                                    }while(wordCount(description) < 2);
```

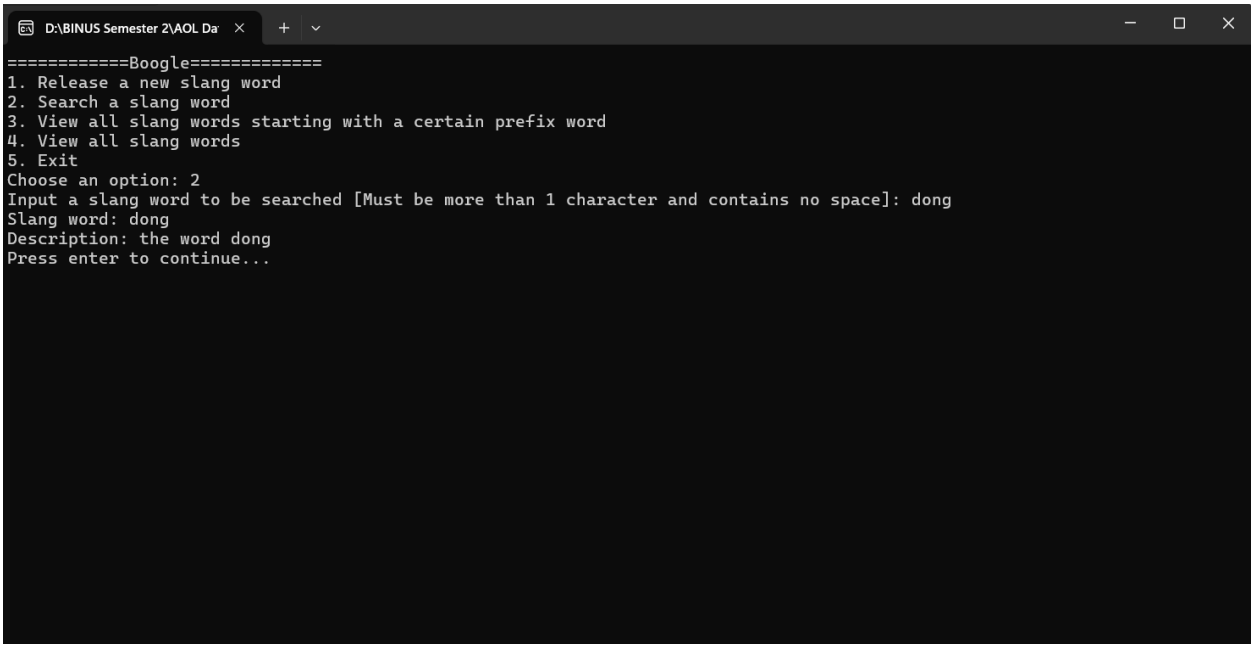
```
            insertWord(&root, word, description);
            break;
        case 2:
            do {
                printf("Input a slang word to be searched [Must be more than 1 character and contains no space]: ");
                scanf("%s", word);getchar();
            } while(strlen(word) <= 1 || haveSpace(word));
                                        exists(root, word);
            break;
        case 3:
            do {
                printf("Input a prefix to be searched: ");
                scanf("%s", word);getchar();
            } while(strlen(word) <= 1 || haveSpace(word));

            searchPrefix(root, word);
            break;
        case 4:
          count = 1;
          printf("List of all the slang words in the dictionary: \n");
            printTrie(root, word, 0, &count);
            break;
        case 5:
            printf("Thank you..., Have a nice day :))\n");
            exit(0);
    }
    printf("Press enter to continue...");
    getchar();
```

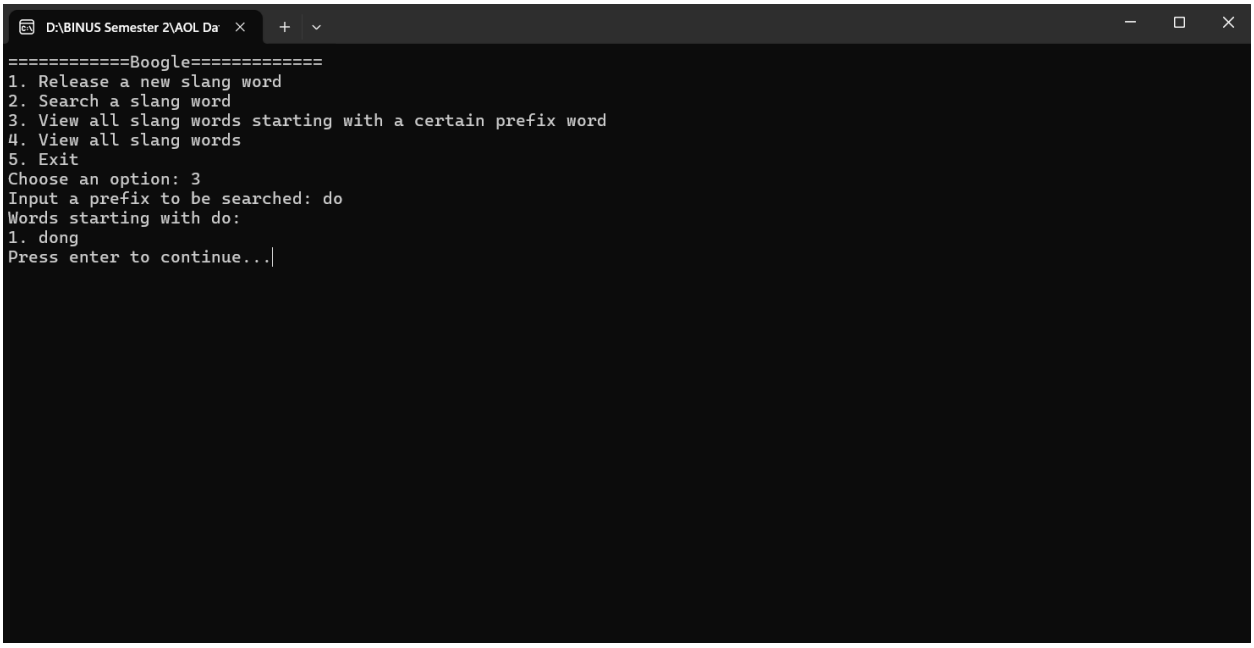
```
    }
        return 0;
}
```

Explanation: This is my main function where my program starts. I first make the menu first and I also create an empty Trie then we enter to while loop where it makes the user to choose an option between insert, search, search prefix or view all and exit) it then performs the chosen option.

Custom Cases:




D:\BINUS Semester 2\AOL Da
==========Boogle==========
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Choose an option: 2
Input a slang word to be searched [Must be more than 1 character and contains no space]: dong
Slang word: dong
Description: the word dong
Press enter to continue...


D:\BINUS Semester 2\AOL Da
==========Boogle==========
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Choose an option: 3
Input a prefix to be searched: do
Words starting with do:
1. dong
Press enter to continue...